```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/
df = pd.read_csv(url)

df.head()
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	mo
0	18.0	8	307.0	130.0	3504	12.0	
1	15.0	8	350.0	165.0	3693	11.5	
2	18.0	8	318.0	150.0	3436	11.0	

Next steps: Generate code with df New interactive sheet

```
df.isnull().sum()
```

	0
mpg	0
cylinders	0
displacement	0
horsepower	6
weight	0
acceleration	0
model_year	0
origin	0
name	0

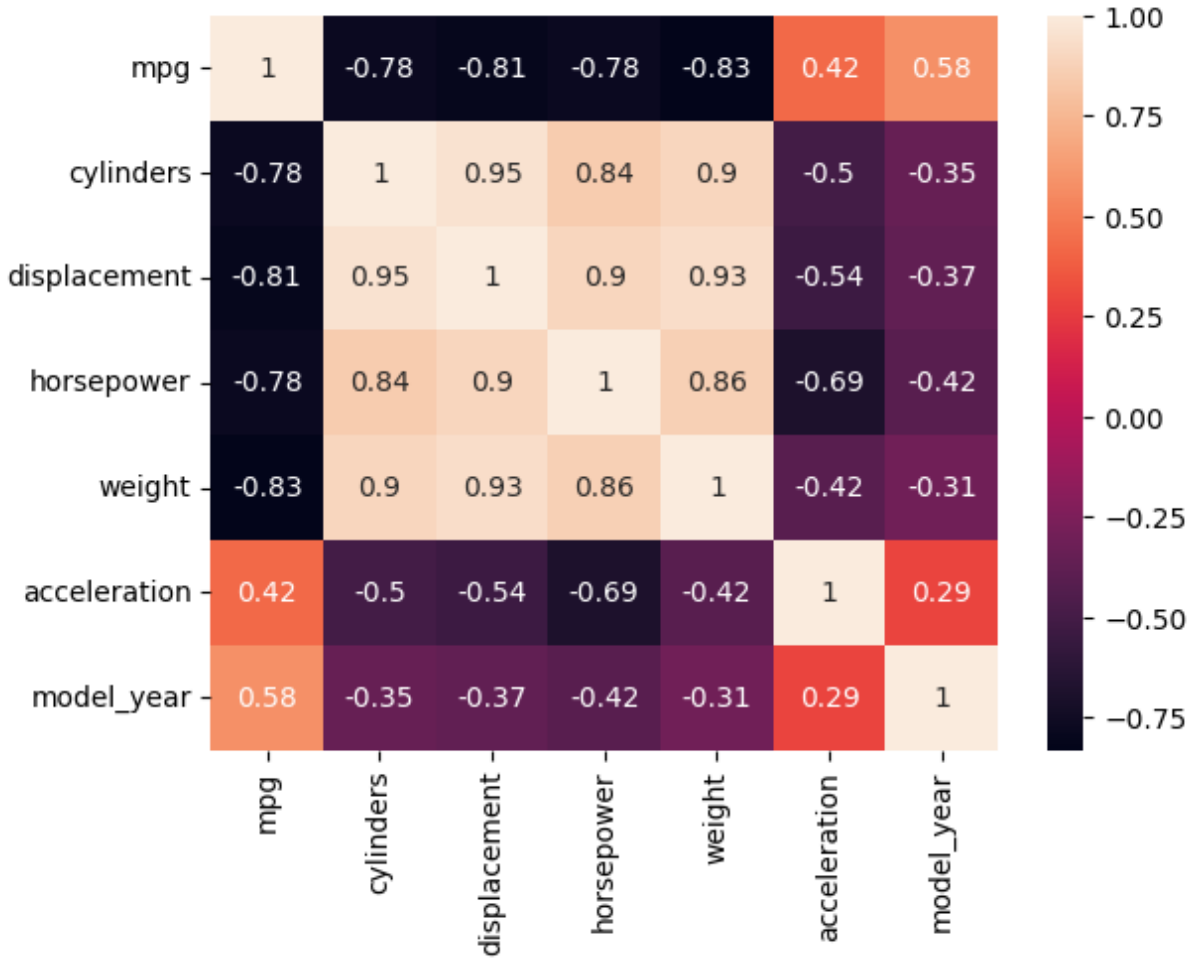
dtype: int64

```
df = df.dropna()
```

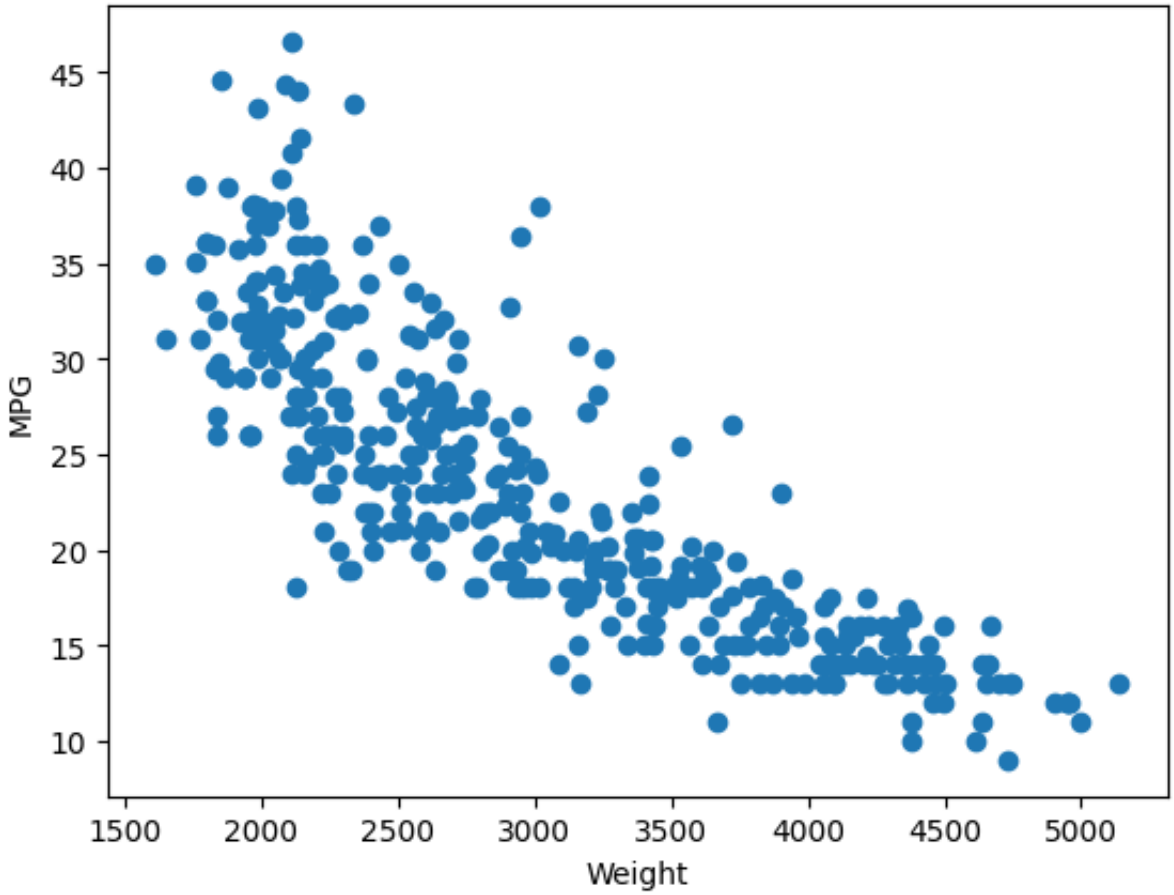
```
print("Skewness:\n", df.skew(numeric_only=True))
print("\nKurtosis:\n", df.kurt(numeric_only=True))
```

```
Skewness:
 mpg             0.457092
cylinders       0.508109
displacement    0.701669
horsepower      1.087326
weight          0.519586
acceleration    0.291587
model_year      0.019688
dtype: float64

Kurtosis:
 mpg            -0.515993
cylinders      -1.398199
displacement   -0.778317
horsepower      0.696947
weight         -0.809259
acceleration    0.444234
model_year     -1.167446
dtype: float64
```

```
plt.figure()
sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.show()
```



```
plt.figure()
plt.scatter(df['weight'], df['mpg'])
plt.xlabel("Weight")
plt.ylabel("MPG")
plt.show()
```




```
df['origin'] = df['origin'].map({
    'usa': 1,
    'europe': 2,
    'japan': 3
})
```

1. Covariance and Pearson Correlation

```
# Covariance
cov_matrix = df[['mpg', 'weight']].cov()
print("Covariance:\n", cov_matrix)

# Pearson Correlation
corr_matrix = df[['mpg', 'weight']].corr()
print("\nPearson Correlation:\n", corr_matrix)
```

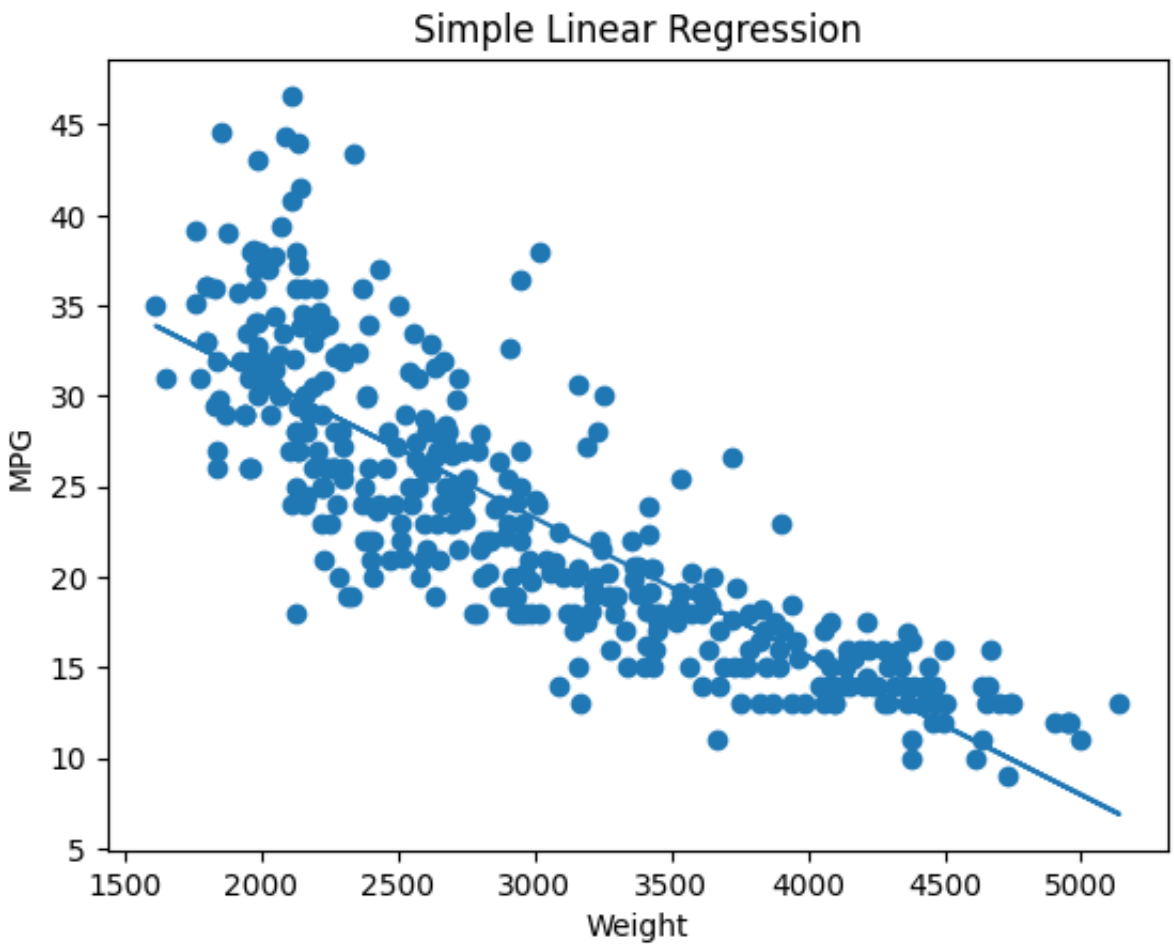
```
Covariance:
                 mpg         weight
mpg       60.918142   -5517.440704
weight -5517.440704  721484.709008

Pearson Correlation:
              mpg    weight
mpg     1.000000 -0.832244
weight -0.832244  1.000000
```

Interpretation:
The covariance is negative and the Pearson correlation coefficient (-0.83) indicates a strong negative relationship between weight and MPG. This means that heavier vehicles tend to have lower fuel efficiency.

2. Simple Linear Regression

```
from sklearn.linear_model import LinearRegression

X = df[['weight']]
y = df['mpg']

model = LinearRegression()
model.fit(X, y)

# Prediction
y_pred = model.predict(X)

# Plot
import matplotlib.pyplot as plt

plt.figure()
plt.scatter(X, y)
plt.plot(X, y_pred)
plt.xlabel("Weight")
plt.ylabel("MPG")
```

```
plt.title("Simple Linear Regression")
plt.show()
```



Interpretation:
The simple linear regression line captures the general downward trend between weight and MPG. However, it only uses one predictor and does not account for other factors such as horsepower or displacement.

3. Multiple Linear Regression

```
X = df[['weight', 'horsepower', 'displacement']]
y = df['mpg']

model_multi = LinearRegression()
model_multi.fit(X, y)

y_pred_multi = model_multi.predict(X)

print("Coefficients:", model_multi.coef_)
print("Intercept:", model_multi.intercept_)
```

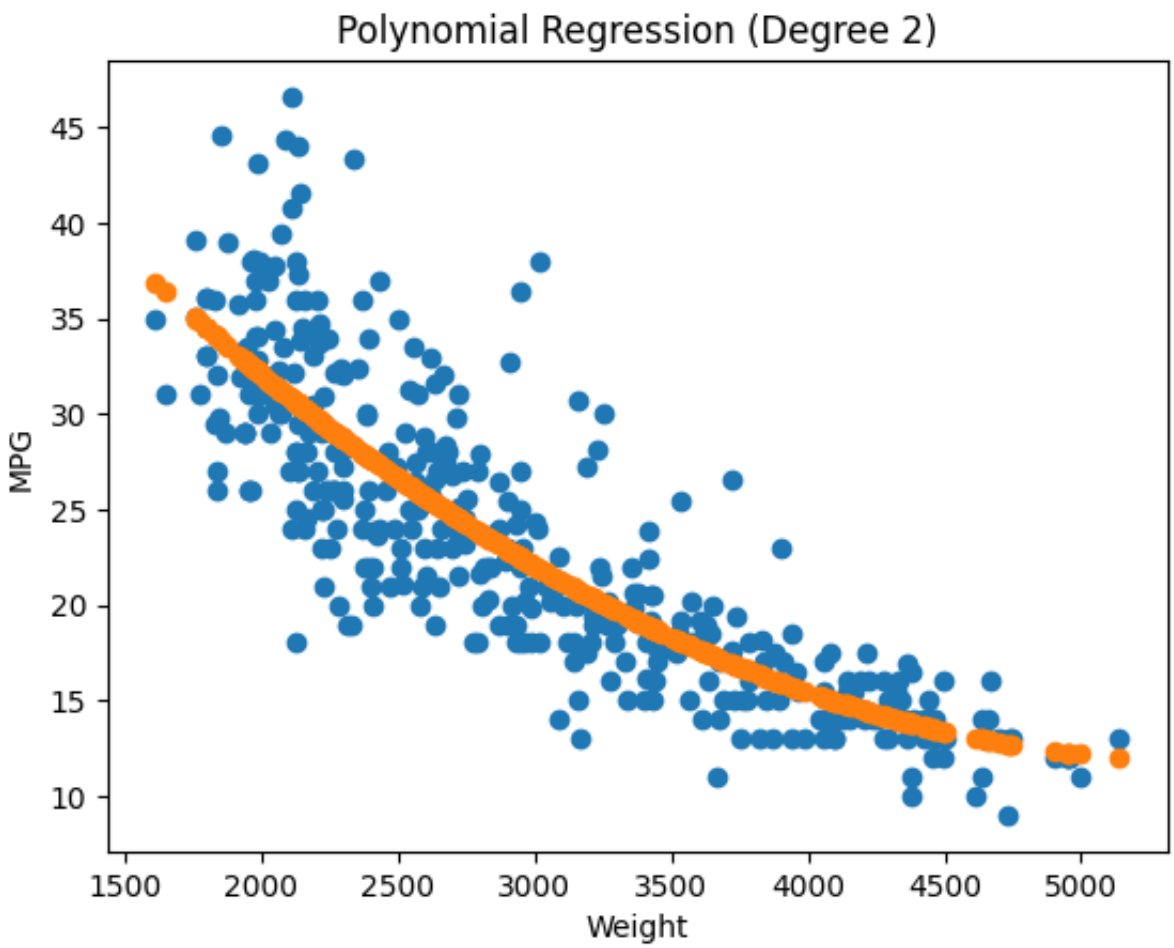
```
Coefficients: [-0.00535159 -0.04167414 -0.00576882]
Intercept: 44.85593569526507
```

Start coding or generate with AI.

Interpretation:
Including multiple predictors improves the explanatory power of the model. However, because some variables are highly correlated, multicollinearity may reduce the stability of the regression coefficients. This reflects real-world scenarios where multiple vehicle characteristics jointly influence fuel efficiency.

4. Polynomial Regression (Degree 2)

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2)

X_poly = poly.fit_transform(df[['weight']])
y = df['mpg']

model_poly = LinearRegression()
model_poly.fit(X_poly, y)

y_pred_poly = model_poly.predict(X_poly)

# Plot
plt.figure()
plt.scatter(df['weight'], y)
plt.scatter(df['weight'], y_pred_poly)
plt.xlabel("Weight")
plt.ylabel("MPG")
plt.title("Polynomial Regression (Degree 2)")
plt.show()
```



Interpretation:
The polynomial model fits the data more closely than the simple linear model, suggesting that the relationship between weight and MPG is not perfectly linear. This improves fit, but also increases the risk of overfitting. This demonstrates that introducing non-linear transformations can capture more complex patterns in the data.

5. Model Evaluation and Comparison

```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squ
import numpy as np
import pandas as pd

# --- Simple Linear Regression ---
r2_simple = r2_score(y, y_pred)
mae_simple = mean_absolute_error(y, y_pred)
rmse_simple = np.sqrt(mean_squared_error(y, y_pred))

# --- Multiple Linear Regression ---
r2_multi = r2_score(y, y_pred_multi)
mae_multi = mean_absolute_error(y, y_pred_multi)
rmse_multi = np.sqrt(mean_squared_error(y, y_pred_multi))

# --- Polynomial Regression ---
r2_poly = r2_score(y, y_pred_poly)
mae_poly = mean_absolute_error(y, y_pred_poly)
rmse_poly = np.sqrt(mean_squared_error(y, y_pred_poly))

# Create comparison table
results = pd.DataFrame({
    'Model': ['Simple Linear', 'Multiple Linear', 'Polynomial (Degr
    'R² Score': [r2_simple, r2_multi, r2_poly],
    'MAE': [mae_simple, mae_multi, mae_poly],
    'RMSE': [rmse_simple, rmse_multi, rmse_poly]
})

results
```

	Model	R² Score	MAE	RMSE
0	Simple Linear	0.692630	3.278702	4.321645
1	Multiple Linear	0.706955	3.231014	4.219738
2	Polynomial (Degree 2)	0.715148	3.052350	4.160338

Next steps: Generate code with `results` New interactive sheet

Interpretation:
The results show that model performance improves slightly as complexity increases. The polynomial regression model achieved the highest R^2 and the lowest MAE and RMSE, but the improvement over multiple linear regression is relatively small. These experiments demonstrate how model performance changes when additional features or non-linear transformations are introduced.